



An Introduction to Computer Science

02

數位資料表示法

在電腦裡，我們需要處理的資料型態包括：數字、文字、語音、音樂、圖形、影像、影片及動畫等，這些資料都會編碼成數位化的資料儲存在電腦裡，等到顯示或列印時，再將數位化的資料解碼成原來得資料格式。數位化的資訊好處多多，它方便我們編輯、處理、儲存，傳輸及播放，以便有效精確地表達意念。

在這一章裡，介紹電腦如何表示數字和文字資料，首先討論各種進位表示法，包括電腦所使用的二進位表示法。接著介紹不同進位表示之間的轉換，包括十進位數與二進位數的互換，以及二進位數與十六進位數的互換。

在整數方面，本章介紹了下列幾種表示法：「無正負符號的整數」、「帶正負符號大小表示法」、「一補數表示法」及「二補數表示法」。其中「二補數表示法」是目前電腦表示整數所用的方法，我們將仔細介紹它的加減法運算原理及計算方式。

在小數方面，IEEE754 標準的浮點數表示法是目前的主流，它主要有三部分：符號位元、指數部分及尾數部分，本章介紹該表示法的數值轉換方式。

另外，我們也介紹通用的字符表示法，包括：ASCII 和 Unicode。在編碼理論方面，常談到的課題還有用來確保資料傳輸正確性的**可偵錯碼** (error-detecting code) 即**可修正碼** (error-correcting)，有興趣的讀者可在找相關資料做進階的研讀。



到底數位是甚麼呢？

數位在電學上是指不連續變化的數量表示法，所謂不連續變化，可以用這個比喻來感受：實數算是一種連續變化的數量表示法，因為任兩數之間總還可以找到第三個數介於它們之間，而且到最後是沒有空隙的，例如 1 和 2 之間，我們可以找到 1.1、1.2、...、1.9 是介於他們之間，而 1.1 和 1.2 之間，我們可以找到 1.11 和 1.12、...。

整數從某總角度來看，它是不連續的數量表示法，例如整數 1 和整數 2 之間，我們再也找不到任何整數是介於他們之間的，不像實數系統總有第三個數將大家串連在一起。

針對不連續變化的數量，可以用位元 (binary digit; bit) 的組合來計數，位元是數位資訊的基本粒子，也是電腦儲存或傳遞資料的最小單位，常用 0 或 1 來表示。電腦會採用位元表示資料，主要是因為電子元件的穩定狀態有兩種：一種是「開」(通常用來表示“1”)及一種是「關」(通常用來表示“0”)。單一的 0 或 1 稱為位元 (bit)，早期電腦以 8 個位元為存取單位，因此 8 個位元稱為位元組 (byte)。

兩個位元可以有 2^2 共 4 種組合 (00, 01, 10, 11)。圖 2-1 列了一到五個位元的各種組合，在這圖中我們注意到：每增加一個位元，我們的組合數就加倍。因此， n 個位元可以有 2^n 種不同的組合，就可用來表示 2^n 種不同物件。

1位元	2位元	3位元	4位元	5位元
0	00	000	0000	00000
1	01	001	0001	00001
	10	010	0010	00010
	11	011	0011	00011
		100	0100	00100
		101	0101	00101
		110	0110	00110
		111	0111	00111
			1000	01000
			1001	01001
			1010	01010
			1011	01011
			1100	01100
			1101	01101
			1110	01110
			1111	01111
				10000
				10001
				10010
				10011
				10100
				10101
				10110
				10111
				11000
				11001
				11010
				11011
				11100
				11101
				11110
				11111

圖 2-1 一到五個位元的各種組合

需要用多少位元才能表示中文字呢？答案是 16 位元。因為 16 位元可以有 2 的 16 次方共 65,536 種組合，遠超過常用的中文字數目，因此 16 個位元可表示中文字。由於網際網路日益普及，為避免各國文字的位元表示方式有所衝突，萬國碼 (Unicode) 就依實現方式不同，而以不同位元個數的組合來公定各國文字。

電腦的存取機制以位元組為基本單位，所以表示資料所需的位元數，通常是 8、16 及 32 等。

數位化的基本定義 (參考資料-浩躍國際)

數位化是一種將資料轉化為計算機可以處理的數字格式的過程，這些數字(通常是二進制數字 0 和 1)可以被儲存、處理和傳送，從而在互聯網、設備和各類應用程式中實現無縫操作。這個轉化過程涉及到多種技術手段和工具，從光纖傳輸數據的網絡基礎設施，到軟體層次的加密技術和數據分析都是不可或缺的一部分。

- **數據儲存**：使用數位媒體如硬碟、SSD 或雲端儲存。
- **數據傳輸**：採用網絡協議和通訊技術如 Wi-Fi、以太網和 5G。
- **數據處理**：包括大數據分析、機器學習和人工智能。

數位是指使用二進位制（即 0 和 1）來表示和儲存信息的方式。這種方式可以用來處理圖像、音頻、文字和其他各種數據。

數位，這個現代生活中的重要議題，不僅改變了我們的日常互動，更深刻地影響了我們的思維方式與未來願景。透過了解數位的基本概念，我們不僅能更好地適應這個不斷變化的時代，也能更積極地參與到數位世界的構建當中。無論科技如何進步，數位所帶來的便捷與挑戰，都提醒著我們要保持開放與學習的心態，迎接每一個新的契機，探索每一個未知的領域。數位的世界無限寬廣，願我們每一個人都能在這個數位時代裡找到屬於自己的光芒與方向。



2-1

資料型態

在電腦裡，我們需要處理的資料型態 (data type) 包括: 數字、文字、語音、音樂、圖形、影像、影片及動畫等，會編碼成位元字串儲存在電腦裡，等到顯示或列印時，再解碼成原來的資料格式 (圖2-2)。

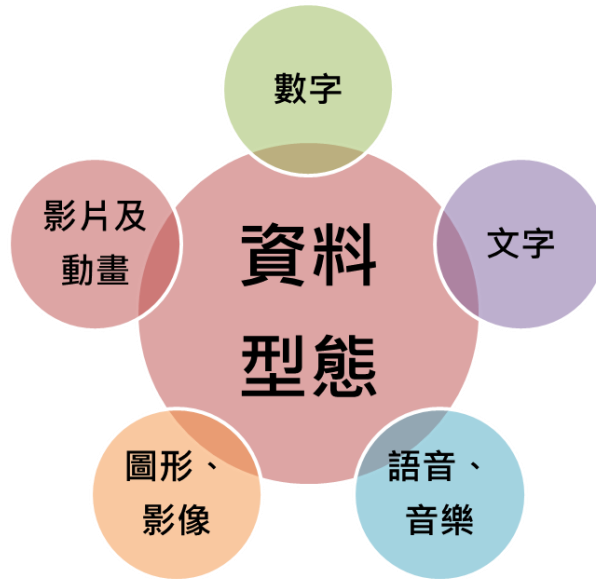


圖 2-2 不同的資料型態都以位元字串儲存在腦裡

當所有的資料型態儲存到電腦時，都被轉換成一致的表達方式，稱為位元樣式。位元〔bit，二進位數字 (binary digit)〕是電腦裡可被儲存的最小資料單元，而且其值為 0 或 1。為了表示不同型態的資料，我們使用位元樣式 (bit pattern)，這是連串位元，又稱為位元字串 (a string of bits)。一個具有 8 位元的位元樣式稱為一個位元組 (byte)。字組 (word) 表示較長的位元樣式。屬於不同資料型態的一筆資料可能在記憶體內儲存成相同的樣式。

資料類型 (參考資料-維基百科)

在程式設計的类型系統中，資料類型 (Data type)，又稱資料型態、資料型別，是用來約束資料的解釋。在程式語言中，常見的資料類型包括原始類型（如：整數、浮點數或字元）、多元組、記錄單元、代數資料類型、抽象資料類型、參考型別、類別以及函式型別。資料類型描述了數值的表示法、解釋和結構，並以演算法操作，或是物件在記憶體中的儲存區，或者其它儲存裝置。

所有在電腦中，基於數位電子學的底層資料，都是以位元 (0 或 1) 表示。其中資料的最小的定址單位，稱為位元組 (通常是八位元，以八個位元為一組)。機器碼指令處理的單位，稱作字長 (至 2007 年止，一般為 32 或 64 位元) 大部分對字長的指令解譯，主要以二進制為主，如一個 32 位元的字長，可以表示從 0 至 232-1 的無符號整數值，或者表示從 -231 至 231-1 的有符號整數值。由於有了二的補數，機器語言和機器大多不需要區分無符號和有符號資料類型。存在著特殊的算術指令，對字長中的位元使用不同的解釋，以此作為浮點數。

2-2 二進位表示法

古代的巴比倫人，所用的數字系統是六十進位法，逢「六十」進一，現在這樣的進位法，除了每分鐘六十秒及每小時六十分外，已不多見。

現今公制是以十為基數，採用十進位法，「十進」集滿十進一。一個數字在不同的位置上所表示的數值也就不同，如三位數“523”，右邊的”3”在個位數上表示3個一，中間的”2”在十位上就表示2個十，左邊的“5”在百位上則表示5個百，換句話說， $523 = 5 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$ 。因為在電腦的電子元件中，最穩定簡單的狀態就是「開」（通常用來表示“1”）與「關」（通常用來表示“0”），所以目前通行的電腦皆以2為基數，用二進位符號來儲存資料。

在電腦系統的資料顯示方面，我們也常使用十六進位數，這是因為一個位元組有八個位元，可正好切成兩個十六進位數，例如：11010011，前面的1101可用一個十六進位數表示，後面的0011也可用一個十六進位數表示。十進位數用十個阿拉伯數字表示，十六進位數就需十六個符號表是各個不同的數字。在十六進位系統裡，0到15的十六個數字分別用阿拉伯數字的0到9及A到F表示（見表2-1）。因此，11010011這個二位元字串，就可表示成D3₁₆或0xD3（x起頭，代表該數為十六進位數）。

二進制（參考資料-維基百科）

現代的二進制記數系統由戈特弗里德·萊布尼茨於1679年設計，在他1703年發表的文章《論只使用符號0和1的二進制算術，兼論其用途及它賦予伏羲所使用的古老圖形的意義》出現。與二進制數相關的系統在一些更早的文化中也有出現，包括古代的埃及、中國、印度以及太平洋島原住民文明。其中，古代中國的《易經》尤其引起了萊布尼茨的聯想。

二進制（binary）在數學和數位電路中指以2為底數的記數系統，以2為基數代表系統是二進位制的。這一系統中，通常用兩個不同的數字0和1來表示。數字電子電路中，邏輯閘直接採用了二進制，因此現代的計算機和依賴計算機的裝置裡都用到二進制。每個數字稱為一個位元（二進制位）或位元（Bit，Binary digit的縮寫）。

表 2-1 十六進位的數字符號及其所對應的十進位及二進位

十進位	二進位	十六進位	十進位	二進位	十六進位
0	0	0	8	1000	8
1	1	1	9	1001	9
2	10	2	10	1010	A
3	11	3	11	1011	B
4	100	4	12	1100	C
5	101	5	13	1101	D
6	110	6	14	1110	E
7	111	7	15	1111	F



2-3

各種進位表示法的轉換

現在我們要討論各種進位表示法的轉換，為了簡化討論，我們以範例的方式來探討 (1) 十進位數與二進位數的互換，以及 (2) 二進位數與十六進位數的互換。

十進位數與二進位數的互換

給定一個二進位數，我們如何將它轉換為十進位呢？我們只要將每個二進位數字和它所對應的 2 的次方項（以十進位表示）相乘即可。

範例1： 10110101.1101_2 所對應的十進位數為 181.8125，其計算步驟如圖 2-3。

$$\begin{array}{cccccccccccc}
 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & . & 1 & 1 & 0 & 1 \\
 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 & & 2^{-1} & 2^{-2} & 2^{-3} & 2^{-4} \\
 \rightarrow & 2^7 & + & 2^5 & + & 2^4 & + & 2^2 & + & 2^0 & + & 2^{-1} & + & 2^{-2} & + & 2^{-4} \\
 = & 128 & + & 32 & + & 16 & + & 4 & + & 1 & + & 0.5 & + & 0.25 & + & 0.0625 \\
 = & 181.8125
 \end{array}$$

圖 2-3 10110101.1101_2 所對應的十進位數為 181.8125

現在讓我們先來看一下整數的部分，我們將這個數值除以 2，可得到餘數為最低為數的數值，再把商數除以 2，可得到餘數為次低位數的數值；以此類推直到商數為 0。

範例2： 現在以 181 為例，181除以2得商數 90，餘數為1，因此我們知道**最小位數值為1**；再將 90 除以2，得商數 45，餘數為 0，因此我們知道**次小位數值為 0**；以此類推。詳細的計算步驟如圖 2-4，我們所得到的二進位數為 10110101_2 。

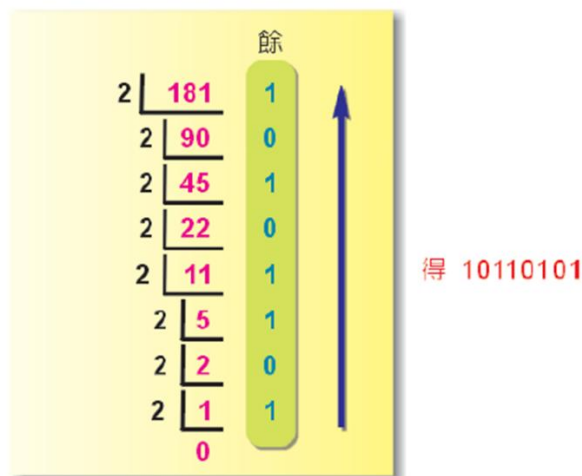


圖 2-4 十進位181所對應的二進位數為 10110101_2

小數部分如何轉換呢？如果將該數值乘以 2，若越過小數點的數值為 0，則最高位數的數值為 0；若為 1，則最高位數的數值為 1。再將小數部分乘以 2，若越過小數點的數值為 0，則次高位數的數值為 0；若為 1，則次高位數的數值為 1。以此類推，直到剩餘的小數部分為 0。

範例3：現在我們以 0.8125 為例，0.8125 乘以 2 得 1.625，越過小數點的整數為 1，因此我們知道**最高位數值**為 1；再將剩下的小數部分 0.625 乘以 2，得 1.25，越過小數點的整數為 1，因此我們知道**次高位數值**為 1；以此類推。詳細的計算步驟如圖 2-5，我們所得到的二進位數為 0.1101_2 。

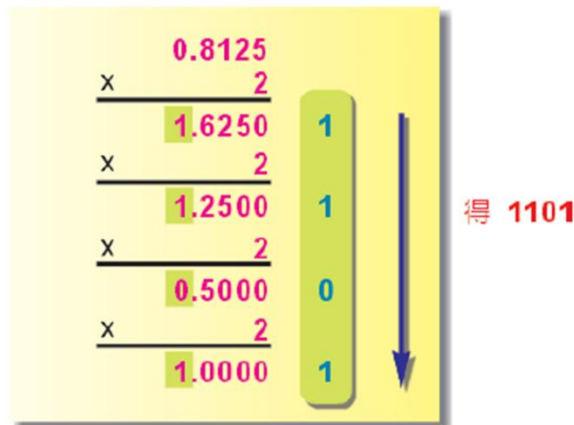


圖 2-5 十進位 0.8125 所對應的二進位數為 0.1101_2

範例4：請問十進位 0.1 的二進位表示法為何？0.1 乘以 2 得 0.2，越過小數點的整數為 0，因此我們知道**最高位數值**為 0。將剩下的小數部分 0.2 乘以 2，得 0.4，越過小數點的整數為 0，因此我們知道**次高位數值**為 0。將剩下的小數部分 0.4 乘以 2，得 0.8，越過小數點的整數為 0，因此我們知道**位數值**為 0。將剩下的小數部分 0.8 乘以 2，得 1.6，越過小數點的整數為 1，因此我們知道**位數值**為 1。將剩下的小數部分 0.6 乘以 2，得 1.2，越過小數點的整數為 1，因此我們知道**位數值**為 1。將剩下的小數部分 0.2 乘以 2，咦？0.2 剛剛不是出現過了嗎？這樣下去似乎沒完沒了了！沒錯！詳細的計算步驟如圖 2-6，我們所得到的二進位數為 $0.000110011\dots_2$ 。

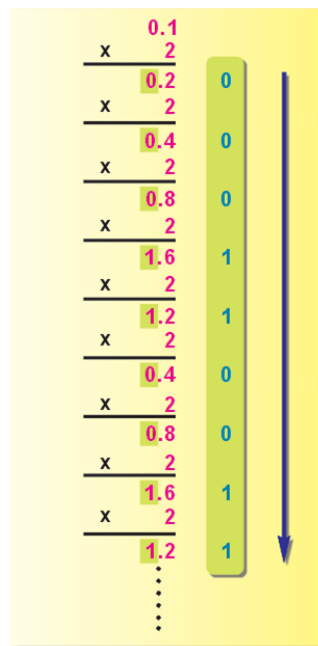


圖 2-6 十進位 0.1 所對應的二進位數為無窮位數 $0.000110011\dots_2$



二進位轉十進位 (網路資料)

將數學問題 (像是 $77+1$) 轉成二進位給電腦做計算，計算結果是不太直觀的二進位表示法，需要轉回我們熟悉的十進位，呈現在螢幕上，我們才能直觀解讀計算結果。我們知道，將 78 轉成二進位是除 2，那要逆回來變成十進位就是乘 2，在介紹如何乘 2 回來之前，我們用另一個角度回顧十進位：

從 (圖 2-7) 可知，十進位制的意思就是以 10 為基底 (即 10^n) 表達數字 $78_{10} = 7 \times 10 + 8 \times 1 = 7 \times 10^1 + 8 \times 10^0$

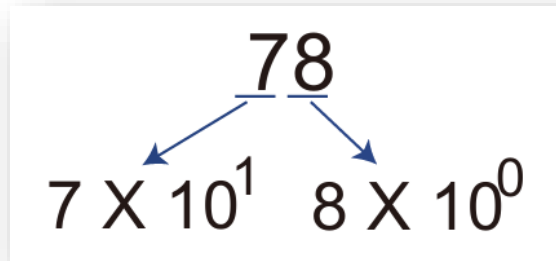


圖 2-7 $70+8$ 等於 78

二進位制就是以 2 為基底表達數字：以 (圖 2-8) 為例，把圖上數字都加起來。

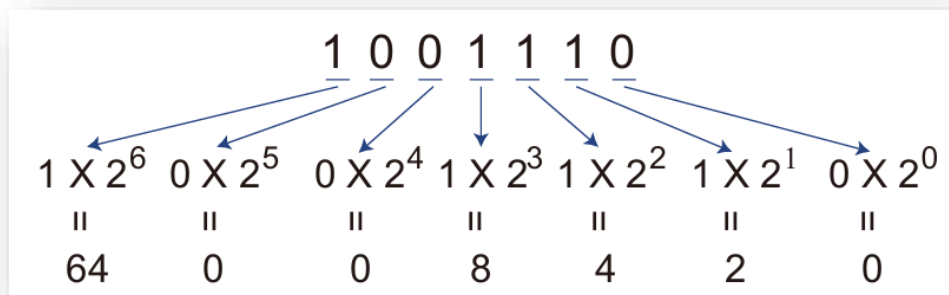


圖 2-8 二進位 1001110 所對應的十進位數為 78

如果是小數點，剛剛是透過乘 2 轉成二進位，那逆回來十進位就要除 2，不過我們可以用更 general 的方法來表達，即 $\times 2^{-n}$ ，以 1001110.101 為例：看粉色的部分，透過除 2 的方式將小數點的部分從二進位轉回十進位

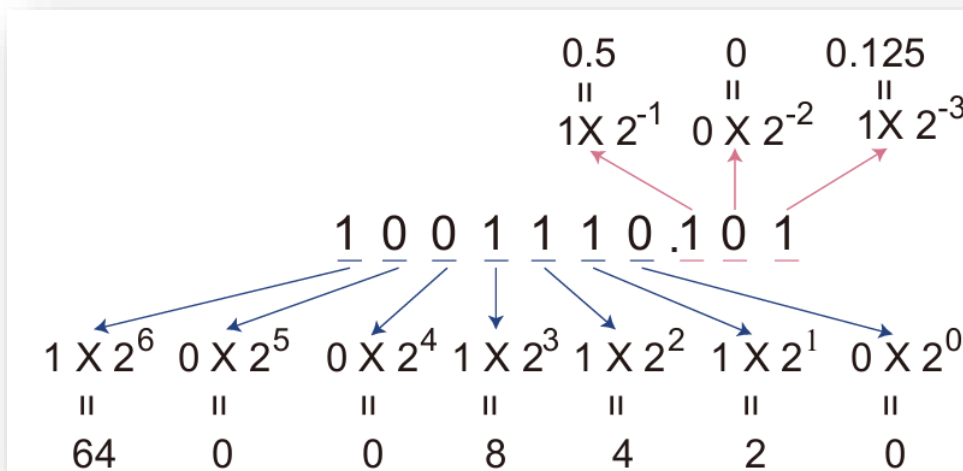


圖 2-9 二進位 1001110.101 所對應的十進位數為 78.625

二進位數要如何轉換為十六進位數呢？若將整數部份從小數點往左每四個分一群；小數部分從小數點往右每四個分一群，則我們可以將**取低位”二進位數”**改寫成以 16 為基數的多項式，如圖 2-10。

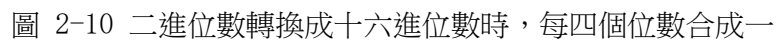


圖 2-11 110110101.11011_2 的十六進位表示法為 $1B5.D8_{16}$

圖 2-12 1B5.D8₁₆ 的二進位表示法為 110110101.11011₂



2-4

整數表示法

介紹過各種進位的表示法及其轉換法後，現在讓我們來談談在電腦裡如何表示一個整數呢？如果只要表示非負的整數即可，則情況比較單純，並不會造成困擾，只要將最小的位元字串（亦即全為 0 的字串）給 0，依序表示到最大的數即可，這樣 n 個位元藉可表示 2^n 個數，所表示的整數範圍為 $0 \sim 2^n - 1$ 。例如，如果使用 8 個位元，則可表示 $0 \sim 2^8 - 1$ 間的所有整數，也就是從 0 到 255 的所有整數。表 2-2 列了位元字串與十進位數的對應表。我們稱這種整數為**無正負號的整數**（unsigned integer）

表 2-2 以8位元所表示的「無正負符號的整數」

位元字串	十進位數
00000000	0
00000001	1
00000010	2
...	...
11111110	254
11111111	255

但負整數的表示就得仔細斟酌，如此各項運算才能順利進行。假設我們用 n 個位元表示一數，則我們可表示的數有 2^n 個，如果要同時表示正數和負數，最直接的做法是採用**帶正負符號大小表示法**（sign-and-magnitude-representation），這方法以位元字串的最左邊的位元當作符號位元，它表示數正負：0為正數；1為負數。剩下的 $n-1$ 個位元就可用來表示數的大小，所以我們以位元 0 開頭的整數範圍為 $0 \sim 2^{n-1} - 1$ ；而以位元 1 開頭的整數範圍為 $0 \sim -(2^{n-1} - 1)$ 。例如，如果使用 8 個位元，則可表示 $-(2^7 - 1) \sim 2^7 - 1$ 間的所有整數，也就是從 -127 到 127 的所有整數。表 2-3 列了位元字串與十進位數的對應表。這種表示法有潛在的兩個問題：第一，它有兩個 0，+0(000...00) 和 -0(100...00)；第二，正數和負數的運算(例如加加減)並不直接(稍後介紹「補數表示法」時會更有所體會)，所以目前的電腦並不採用這種方式表示整數。

目前電腦儲存整數的標準方式是採用**二補數表示法**(two's complement representation)，補數的概念是只要補多少才滿。

表 2-3 以8位元所表示的「帶正負符號大小表示法」

位元字串	十進位數
00000000	0
00000001	1
...	...
01111111	127
10000000	-0
10000001	-1
...	...
11111111	-127

一補數表示法

談二補數表示法之前，我們先看看較簡單的一補數表示法 (one's complement representation)。一補數表示法比較單純一點，「正數」的表示方式和前面提到的「帶正負符號的整數」一樣，而負數的表示法會有點不同。「負數」的表示會將二進位數的每個數字做反轉，0 轉成 1，1 轉成 0，轉換後得到的數就稱為原二進位的一補數，給定一個十進位數值，轉換成它的個補數表示法步驟如下：

步驟1： 先忽略其符號，將數字的部分轉成二進位數值。

步驟2： 若二進位數值超過 $n-1$ 個位元，則為「溢位」(overflow)，無法進行轉換；否則在它的左邊補 0，直到共有 n 個位元為止。

步驟3： 若所要轉換的數為正數或零，則步驟 2 所得數值即為所求；若為負數，則將每個位元做補數轉換(原0轉1；原1轉0)。

假設我們以 8 位元表示一個數，讓我們看看幾個例題。

範例7： 41的一補數表示法為何？第一步先將41轉成二進位數值101001；第二步在二進位數值左邊補上0，使00101001共有8個位元，因為要表示的數為正數，所以00101001即為所求。

範例8： -41的一補數表示法為何？第一步先將41轉成二進位數值101001；第二步在二進位數值左邊補上0，使00101001共有8個位元，這時所要表示的數為負數，所以將原為 0 的轉成 1；原為 1 的轉成 0，得 11010110。圖 2-13 描繪這個轉換程序。

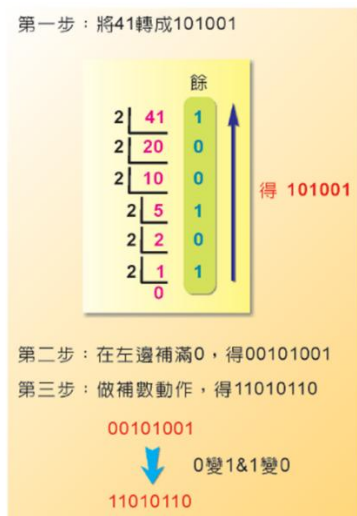


圖 2-13 -41的八位元一補數表示法為11010110

反之，「11010110」這個一補數所表示的值為多少呢？讀者若再多觀察一下當初轉換成一補數的步驟，應可反向推導出下面的法則：

如果最左邊的位元是 0，則表示該數是正數，只要將後面的位元以前面介紹的二進位轉時進位方式求出其數值即可。

如果最左邊的位元是 1，則表示該數是負數，先將每個位元做補數轉換，原為 0 的轉成 1，原為 1 的轉成 0。然後將該二進位轉成十進位，再加上一個負號即可。

範例9： 給定一補數 11010110，因為最左邊位元是1，所以我們先將這補數原 0 的轉成 1；原 1 的轉成 0，得 00101001。再將二進位的 00101001 轉成十進位的 41，然後加上一個負號，得 -41。

給定一個一補數，採用之前介紹之一補數轉換第三步負數部分的補數動作求取它的一補數，這樣所得的數，在求取它的一補數，所得的數仍為自己。換句話說，一個數的一補數的一補數仍自己。舉例而言，41 的一補數表示法為 00101001，該補數的一補數為 11010110(-41)，而這個一補數的一補數為 00101001，又回到 41。再看一個例子，-41 的一補數表示法為 11010110，該補數的一補數為 00101001(41)，而這個一補數的一補數為 11010110，又回到 -41。

一補數表示法以位元字串的最左邊的位元當作符號位元，它表示數的正負：0 為正數；1 為負數。剩下的 $n-1$ 個位元就可用來表示數的大小，所以我們以位元 0 開頭的整數範圍為 $0 \sim 2^{n-1}-1$ ；而以位元 1 開頭的整數範圍為 $0 \sim -(2^{n-1}-1)$ 。例如，如果使用 8 個位元，則可表示 $-(2^7-1) \sim 2^7-1$ 間的所有整數，也就是從 -127 到 127 的所有整數。

一補數法也碰到「兩個 0」的問題，以八元為例，00000000 和 11111111 都是 0，這會造成計算上的困擾，再加上它的加減法也不是那麼直接，所以它並非目前電腦用來表示整數所用的方式。

二補數表示法

「二補數表示法」是目前電腦表示整數所用的方法，如前面所言，補數方法係以位元字串最左邊的位元當作符號位元，以它來表示數的正負：0 為正數；1 為負數；其餘的 $n-1$ 個位元則用來表示正負符號外的數值大小。給定一個十進位數值，轉換成它的二補數表示法步驟如下：

步驟1： 先忽略其符號，將數字的部分轉成二進位數值；

步驟2： 若該二進位數值超過 $n-1$ 個位元，則為溢位(overflow)，無法進行轉換；否則在它的左邊補 0 直到共有 n 個位元為止。若所轉換的數為 -2^{n-1} ，則轉出來的數為 1 後面有 $n-1$ 個 0，此即為該數的二補數表示法，不應視為溢位。

步驟3： 若所轉換的數為正數或零，則上步所得數值即為所求；若為負數，則最右邊的那些 0 及最右邊的第一個 1 保持不變，將其餘位元做補數轉換（原 0 轉 1；原 1 轉 0）。

假設我們已 8 位元表示一個數，讓我們看看幾個例題。

範例10： 40的二補數表示法為何？第一步先將 40 轉成二進位數值 101000；第二步在二進位數值左邊補上 0，使得 00101000 共有 8 個位元，因為要表示的數為正數，所以 00101000 即為所求。

範例11： -40的二補數表示法為何？第一步先將 40 轉成二進位數值 101000；第二步在二進位數值左邊補上 0，使得 00101000 共有 8 個位元；這時所要表示的數為負數，所以最右邊的三個 0 及第一個 1 維持不變，其餘的將原為 0 的轉成 1；原為 1 的轉成 0，得 11011000。

反之，「11011000」這個二補數所表示的值為多少呢？如同一補數反推的作法，讀者若再多觀察一下當初轉換成二補數的步驟，應可反向推導出下面的法則：

如果最左邊的位元是 0，則表示該數是正數，只要將後面的位元以前面介紹的二進位轉十進位方式求出其數值即可。

如果最左邊的位元是 1，則表示該數是負數，保留最右邊的那些 0 及最右邊的第一個 1；將其餘的位元做補數轉換，原為 0 的轉成 1；原為 1 的轉成 0。然後將該二進位轉成十進位，

再加上一個負號即可。

範例12： 給定二補數 11011000，因為最左邊位元是1，所以我們先保留最右邊的那三個 0 及最右邊的第一個 1，再將其他的位元原為 0 的轉成 1；原為 1 轉成 0，得 00101000。再將二進位的 00101000 轉成十進位的 40，然後加上一個負號得 -40。

給定一個二補數，採用之前介紹之二補數轉換第三步負數部分的補數動作求取它的二補數，這樣所得的數，在求取它的二補數，所得的數仍為自己。換句話說，一個數的二補數的二補數仍自己。舉例而言，40 的二補數表示法為 00101000，該補數的二補數為 11011000(-40)，而這個二補數的二補數為 00101000，又回到 40。再看一個例子，-40 二補數表示法為 11011000，該補數的二補數為 00101000(40)，而這個二補數的二補數為 11011000，又回到 -40。

二補數表示法以位元字串的最左邊的位元當作符號位元，它表示數的正負：0 為正數；1 為負數。剩下的 $n-1$ 個位元就可用來表示數的大小，所以我們以位元 0 開頭的整數範圍為 $0 \sim 2^{n-1}-1$ ；而以位元 1 開頭的整數範圍為 $-1 \sim -2^{n-1}$ 。例如，如果使用 8 個位元，則可表示 $-2^7 \sim 2^7-1$ 間的所有整數，也就是從 -128 到 127 的所有整數。

二補數的 0 只有一個，以八元為例，就是 00000000。如圖 2-14 列了二補數表示法位元字串與數值的對應關係。

01111111	127
01111110	126
.	.
.	.
.	.
00000010	2
00000001	1
00000000	0
11111111	-1
11111110	-2
.	.
.	.
.	.
10000010	-126
10000001	-127
10000000	-128

圖 2-14 八位元二補數表示法的位元字串與數值之對應

二補數的加減法

二補數的加法很容易進行，先將所加的兩個數之二補數位元對齊，從最右邊的位元開始加起，因為是二進位，所以若相對位置的位元相加為二或以上，則有進位。若有進位，則往左邊傳遞；若最左邊的位元相加有進位，則忽略這個進位。此外，當兩個正數相加後，若結果的最左邊符號位元變成 1，則有溢位(overflow)。當兩個負數相加後，若結果的最左邊符號位元變成 0，則有溢位(overflow)。在談它的運作原理前，讓我們先以例子來看看它是如何進行的。

範例13： 最簡單的情況是兩個正數相加，圖 2-15 為兩個正數相加的過程。

					1					← 進位
	16		0	0	0	1	0	0	0	0
+)	24		0	0	0	1	1	0	0	0
	40		0	0	1	0	1	0	0	0

圖 2-15 二補數表示法的兩正數相

範例14：圖 2-16 是一個正數加上一個負數，且結果為正的例子。在這過程中，最左邊的位元相加有進位，我們對那個進位忽略不管。

忽略

1 1 1

← 進位

-16	1	1	1	1	0	0	0	0
24	0	0	0	1	1	0	0	0
8	0	0	0	0	1	0	0	0

圖 2-16 二補數表示法的一正一負相加，且結果為正

範例15：圖 2-17 是一個正數加上一個負數，且結果為負的例子。在這過程中，最左邊的位元相加並沒有進位。

	-24	1	1	1	0	1	0	0	0
+)	16	0	0	0	1	0	0	0	0
	-8	1	1	1	1	1	0	0	0

圖 2-17 二補數表示法的一正一負相加，且結果為負

範例16： 圖 2-18 是兩個負數相加。在這過程中，最左邊的位元相加有進位，我們對那個進位忽略不管。

		忽略	1	1	1					← 進位
	-16		1	1	1	1	0	0	0	0
+)	-24		1	1	1	0	1	0	0	0
	-40		1	1	0	1	1	0	0	0

圖 2-18 二補數表示法的兩負數相加

範例17： 圖 2-19 是兩個正數相加超過儲存範圍的例子，我們稱之為**溢位**(overflow)。在這過程中，最左邊的符號位元相加結果變成 1，在二補數表示法代表負數，也就是兩正數相加反變成負數，其原因乃 n 個位元的二補數最大正數為 $2^{n-1}-1$ ，當 $n=8$ 時，最大正數為 127，而在此我們的結果為 129，已超過正數儲存範圍。

		1	1	1	1					← 進位
	120		0	1	1	1	1	0	0	0
+)	9		0	0	0	0	1	0	0	1
	129		1	0	0	0	0	0	0	1
		溢位								

圖 2-19 二補數表示法的兩正數相加結果超過正數儲存範圍

範例18： 圖 2-20 是兩個負數相加超過儲存範圍的例子，我們稱之為**溢位**(overflow)。在這過程中，最左邊的符號位元相加結果變成 0，在二補數表示法代表正數，也就是兩負數相加反變成正數，其原因乃 n 個位元的二補數最小負數為 $-2^{n-1}-1$ ，當 $n=8$ 時，最小負數為 -128，而在此我們的結果為 -129，已小於負數儲存範圍。

		忽略	1							← 進位
	-120		1	0	0	0	1	0	0	0
+)	-9		1	1	1	1	0	1	1	1
	-129		0	1	1	1	1	1	1	1
		溢位								

圖 2-20 二補數表示法的兩負數相加結果小於負數儲存範圍

現在讓我們來談談為何二補數的加法可以這樣進行，如果兩個數都是正數，其做法和一般表示法無異，若有進位到最左邊的符號位元，則為溢位。若牽涉到負數的加法，情況就比較複雜了，讓我們先回想一下二補數的一個負數所代表的意義。假設我們以 n 個位元表示一個數，則一個二補數負數 $-x$ 會表示成怎樣的二位元字串呢？

先以例子來體會這個問題，前面我們提到的 40 的二位元字串為 00101000，而 -40 的二補數字串是 11011000，若把最左邊的位元符號也視為數值的一部分，直接把這個二進位字串換成十進位，可得值 216，而這個值正好是 2^8-40 ，也就是 2^8-40 。再看一個例子，24 的二位元字串為 00011000，而 -24 的二補數字串是 11101000，若把最左邊的位元符號也視為數值的一部分，直接把二進位字串換成十進位，可得值 232，而這個值正好是 2^8-24 ，也就是 2^8-24 。這是巧合嗎？

先想想看 2^8 的二進位字串是甚麼呢？正好是一個 1 後面跟著八個 0：100000000。以前面的 -40 為例，它的二補數字串是 11011000，而 2^8-40 又是甚麼東東呢？圖 2-21 描繪了這個相減的過程，當你仔細觀察 2^8-40 的結果和 40 之間的異同時會發現，最右邊的那些 0 和最右邊的第一個 1 維持不變，其他的位元則由 0 變 1 及 1 變 0。為甚麼最右邊的那些 0 會維持不變？因為 2^8 是由一個 1 再加上一長串的 0，所以減的時後並不會改變 40 最右邊的那些 0，為甚麼最右邊的 1 會維持不變？最右邊的 1 是第一個產生借位的地方，因為是二進位，向前一位借來的在這一位算來是 2，再減去 1，仍然得到 1。為甚麼其他的位元由 0 變 1 及 1 變 0 呢？在最右邊的 1 借位後，以這個例子而言， 2^8 左邊的 10000 已被借走 1，所以剩下 1111，由這種全為 1 的字串去減另一個字串，原來為 0 的位元，結果為 1；原來為 1 的位元，則互相抵銷變為 0，因此其他的位元會由 0 變 1 及 1 變 0。



圖 2-21 -40 的二補數表示法正好是 2^8-40

我們很容易就可將這樣的結果由 8 位元擴充到 n 位元，我們得到這樣的結論：一個二補數負數 $-x$ 所表示成的二位元字串其數值為 2^n-x 。

有了這些基本知識，現在讓我們來解釋二補數的加法原理。兩個正數相加的情況與一般加法相同，因此只要考慮有負數的情況及可。令 x 和 y 為兩正數，我們現在考慮 x 加 $-y$ ，也就是一正一負相加的情況，如同前面所說， $-y$ 的二補數表示法之數值為 2^n-y ，這有三種狀況：

狀況1： $x > y$ ，此時所加結果 $x-y$ 應為正數，當我們用二補數相加時，得到 $x+(2^n-y)=2^n+(x-y)$ ，注意在此的 2^n 就會造成最左邊忽略掉的進位，故得到 $x-y$ 。

狀況2： $x = y$ ，此時所加結果 $x-y$ 應為 0，當我們用二補數相加時，得到 $x+(2^n-y)=2^n+(x-y)$ ，注意在此的 2^n 就會造成最左邊忽略掉的進位，故得到 $x-y=0$ 。

狀況3： $x < y$ ，此時所加結果 $x-y$ 應為負數，所以其值為 $-(y-x)$ ，當我們用二補數相加時，得到 $x+(2^n-y)=2^n-(y-x)$ ，注意在此的 2^n 並不會造成最左邊忽略掉的進位，想想看 $-(y-x)$ 的二補數表示法之數值應為甚麼呢？它正好就是 $2^n-(y-x)$ ！

我們現在考慮 $-x$ 加 $-y$ ，也就是兩負數相加的情況，同前面所說， $-x$ 的二補數表示法之數值為 $2^n - x$ ，而 $-y$ 的二補數表示法之數值為 $2^n - y$ ，所以以二補數表示法進行 $-x$ 加 $-y$ 時，得到 $(2^n - x) + (2^n - y) = 2^n + (2^n - (x + y))$ ，這裡的第一個 2^n 是我們忽略掉最左邊的進位，而其餘的部分 $2^n - (x + y)$ ，正好是 $-(x + y)$ 的二補數表示法！

我們已解釋了二補數表示法的加法過程，讀者也許要問，二補數的減法如何進行呢？做法很簡單，只要將所要減去的數，求取它的負值表示法（也就是將該數的二補數表示法，依之前介紹之二補數轉換第三步負數部分的補數動作求取它的二補數，正數變負數，而負數則變正數），再用加法進行及可，例如， $24 - 16$ ，可先求 -16 的二補數表示法，再和 24 相加及為所求。

2-5 浮點數表示法

「浮點數表示法」是電腦表示「實數」最常用的方式，類似十進位的科學記號表示法，目前所採用的「浮點數表示法」都是以 IEEE754 為運算標準，之所以稱為「浮點數表示法」，是因其小數點是浮動的，在有限位元的情況下，我們所能表示的數值範圍比起固定小數點（定點數）還要大以及彈性許多，依位元數可以分為「半精度浮點數」、「單精度浮點數」、「雙精度浮點數」。

給定一實數，第一步先做標準化的動作，這和科學記號做法一樣，例如： 10110.100011 會先轉換成 1.0110100011×2^4 ，因為是二進位的關係，在這裡的小數點左邊的那個數值一定是 1，其餘小數點右邊的 0110100011 稱為**尾數** (mantissa)，而**指數** (exponent) 為 4。



圖 2-22 浮點數表示法的位元欄位功能分配

目前所採用的浮點數表示法以 IEEE 754 標準為主，它主要有三部分：**符號位元** (sign bit)、指數部分及尾數部分。圖 2-22 列了**單倍精準數** (single precision；32 位元) 及**雙倍精準數** (double precision；64 位元) 的位元欄位功能分配。在單倍精準數裡，我們以 1 個位元表示符號；8 個位元表示指數；23 個位元表示尾數部分。而在雙倍精準數裡，我們以 1 個位元表示符號；11 個位元表示指數；52 個位元表示尾數部分。

讓我們以單倍精準數為例，詳細說明它的儲存方式，標準化後的三部分（符號位元、指數部分及尾數部分）之格式簡要說明如下：

- **符號位元**：1 個位元，以 0 表示正數；以 1 表示負數。
- **指數部分**：8 個位元，以過剩 127 (Excess 127) 方式表示，這方式以偏差值 127 來儲存。8 個位元所存的數值可從 0~255，共有 28 種變化。若要儲存正指數和負指數，需做一些統整，過剩 127 方式是將位元數減去 127 所得的值，才是真正所儲存的值。例如：若位元數值為 150，則其所存的數值為 $150 - 127 = 23$ ；若位元數值為 100，則其所存的數值為 $100 - 127 = -27$ 。如此，我們就可表示 -127 到 +128 的所有整數值，其中保留 -127 和 +128 做為特殊用途。
- **尾數部分**：23 個位元，從標準化的小數點後開始存起，不夠的位元部分補 0。

範例19：給定一實數 10110.100011，先轉換成 1.0110100011×2^4 ，因為是正數，故符號位元為 0，尾數部分為 0110100011，指數部分為 4，以過剩 127 方式儲存，必須先加上 127，得 131，再將 131 轉換成二進位，得 10000011。因此 10110.100011 若按 IEEE 754 標準儲存，為 01000001101101000110000000000000。

範例20：-0.0010011 又是如何儲存呢？先轉換成 -1.0011×2^{-3} ，因為是負數，故符號位元為 1，尾數部分為 0011，指數部分為 -3，以過剩 127 方式儲存，必須先加上 127，得 124，再將 124 轉換成二進位，得 01111100。因此 -0.0010011 若按 IEEE 754 標準儲存，為 10111110000110000000000000000000。

範例21：在 IEEE 754 標準下，01000010100101000110000000000000 所儲存的數值為多少呢？首先，位元符號為 0，所以是正數，指數部分是 10000101，換成十進位為 133，再減去 127，得 6，因此，01000010100101000110000000000000 所儲存的數值為 1.0010100011×2^6 ，也就是 1001010.0011。

範例22：在 IEEE 754 標準下，10000010100101000110000000000000 所儲存的數值為多少呢？首先，位元符號為 1，所以是負數，指數部分是 00000101，換成十進位為 5，再減去 127，得 -122，因此，10000010100101000110000000000000 所儲存的數值為 $-1.0010100011 \times 2^{-122}$ 。

因此標準化後，小數點得左邊為 1，所以 IEEE 754 小數點左邊的 1 不儲存，但這時你也許要問，0 是如何儲存的？0 的公定表示法為 00000000000000000000000000000000；而 10000000000000000000000000000000 也是 0（代表 -0）。因為如果假設小數點左邊為 1，再怎樣改變其它位元，也不可能為 0 啊，為了應付此類的問題，指數部分的 -127（00000000）和 +128（11111111）做為特殊用途。例如：如果指數部分全為 0，而尾數部分不為 0，則這樣的數代表著小數點左邊為 0 的位標準化實數；若指數全為 1，則代表著無窮大及其他數字資訊。

除了這些特殊指數部分的狀況外，單倍精準數所能表示的數字範圍有多大呢？最小的正數為 00000001000000000000000000000000，其數值為 $+2^{-126}$ ；最大的正數為 01111110111111111111111111111111，其數值為 $(2-2^{-23}) \times 2^{127}$ 。同理，最大的負數為 10000001000000000000000000000000，其數值為 -2^{-126} ；最小的負數為 11111110111111111111111111111111，其數值為 $-(2-2^{-23}) \times 2^{127}$ 。因此，單倍精準數所能表示的正數從 $+2^{-126}$ 到 $(2-2^{-23}) \times 2^{127}$ ；而負數則從 $-(2-2^{-23}) \times 2^{127}$ 到 -2^{-126} 。

想想看 32 位元最多可表示多少個數呢？最多也只有 2^{32} 個，但上面的數值範圍遠遠超過 2^{32} ，雖然我們損失了數值的解析度，但可彈性地表達了更大範圍的數，IEEE 754 標準表示法還是頗有鑽頭的。

雙倍精準數的方式也很類似，所表達的數值範圍又大了許多！若讀者們對這方面的資訊感興趣，可在網路搜尋 IEEE 754 相關資訊，一定會很有斬獲的。

2-6

ASCII及Unicode

在電腦裡，所有的文字也存成位元字串，因此我們必須有公定的對照表，以便我們能在儲存時將文字轉成位元字串，而在解讀時能將位元字串轉回文字。ASCII (唸成 Ass-key; American Standard Code for Information Interchange; 美國國家資訊交換標準碼) 是當今最普及的公定標準，這標準由美國國家標準局在 1963 年時發表。它以 7 個位元儲存一字符，共有 $2^7=128$ 種組合，想想看，英文字母只有 26 個，若包含大小寫也才 52 個，再加上 10 個阿拉伯數字，以及加減乘除符號，也才 66 個，還有很多組合可用來儲存其餘的標點符號及控制符號等。

標準的 ASCII 只用 7 個位元儲存一個字符，而電腦的儲存常用的位元組為 8 個位元，這多出來的一個位元，有時就用來儲存錯誤檢驗位元 (parity bit)。另外，也有些擴充型的 ASCII 用 8 個位元儲存一字符，這樣共有 $2^8=256$ 種組合，比標準的 ASCII 還多一倍，多出來的組合，我們通常用來儲存非英文的符號、圖形符號及數學符號等。表 2-4 列了 ASCII 基本的符號對照表。

編號 1~31 是一些控制碼，例如：叫聲 (BEL、ASCII 編號 7) 及跳行 (LF、ASCII 編號 10) 等，在此我們省略一些控制碼；編號 32~126 是可印出的符號；編號 127 是刪除控制碼 (DEL)。

表 2-4 ASCII 符號對照表

ASCII 碼	鍵盤	ASCII 碼	鍵盤	ASCII 碼	鍵盤	ASCII 碼	鍵盤
0	NUL	7	BEL	10	LF	13	CR
27	ESC	32	SPACE	33	!	34	? .
35	#	36	\$	37	%	38	&
39	?	40	(	41	)	42	*
43	+	44	,	45	-	46	.
47	/	48	0	49	1	50	2
51	3	52	4	53	5	54	6
55	7	56	8	57	9	58	:
59	;	60	<	61	=	62	>
63	?	64	@	65	A	66	B
67	C	68	D	69	E	70	F
71	G	72	H	73	I	74	J
75	K	76	L	77	M	78	N
79	O	80	P	81	Q	82	R
83	S	84	T	85	U	86	V
87	W	88	X	89	Y	90	Z
91	[	92	\	93	]	94	^
95	_	96	?	97	a	98	b
99	c	100	d	101	e	102	f
103	g	104	h	105	i	106	j
107	k	108	l	109	m	110	n
111	o	112	p	113	q	114	r
115	s	116	t	117	u	118	v
119	w	120	x	121	y	122	z
123	{	124		125	}	126	~
127	DEL						

這些年，當電腦逐步地將全球化成一體，對文字編碼上的需求，已不僅僅是單一英文而已，還須融入各種不同文化的文字等，而這已不是 8 位元（256 種組合）所能表達的，因此，在二十多年前以兩個位元組（共十六位元）進行文字編碼的 Unicode 應運而生。

Unicode 即一般俗稱「萬國碼」的字符編碼標準，中國大陸稱之為「統一碼」。由美國萬國碼制訂委員會於 1988-1991 年間訂定，已成為 ISO 認證之標準（ISO10646），且發展出多種編碼方式：UTF-8、UTF-16 及 UTF-32 等，分別以 8 位元、16 位元及 32 位元為基本單元的編碼方式，UTF-8 在全球資訊網最通行，UTF-16 為 JAVA 及 Windows 所採用，而 UTF-32 則為一些 UNIX 系統使用，前面 128 個符號為 ASCII 字符，其餘則為英、中、日、韓文以及其他非英語系國家之常用文字。

在 Unicode 中，最大宗的分類就是 CJK，這主要是中文、日文及韓文之漢字集。

表 2-5 Unicode 符號對照表

範圍	代表的字符群
0000-007F	基本拉丁字符（與 ASCII 相同）
0080-024F	擴充的拉丁字符
0370-03FF	希臘字符
0E00-0E7F	泰文
0E80-0EFF	寮文
2200-22FF	數學符號
2500-25FF	方塊圖形及幾何圖形
3040-30FF	平假名及片假名
4000-9FFF	CJK；中文、日文及韓文之漢字

除了 ASCII 和 Unicode 外，IBM 的 EBCDIC（唸成 eb' see`dik；Extended Binary Coded Decimal Interchange Code）也是某些機型上常用的編碼方式。國際標準局（ISO）用四個位元組（也就是 32 位元）制定一種編碼方式，可以有 232 種組合，這樣就可表示多達 4,294,967,296 種字符。

最後，我們再稍稍補充說明中文字體的編碼，以正體字而言，大五碼（Big5；約一萬六千字）是廣受歡迎的一種編碼方式，盛行於台灣及香港。以簡體字而言，國標（GB；約八千字）是廣受歡迎的編碼方式，盛行於大陸地區。這些字體以逐步被包含於 Unicode 的 CJK 字集中，未來的整合一致化指日可待。

電腦是使用編碼系統將文數字等資料轉換成「二進位」，常見的編碼系統有 ASCII, Unicode, EBCDIC（IBM 的）等。一個字符所代表的位元依照不同編碼系統而不同。不論在哪個程式語言和機器中，也不論用著什麼母語，**每個字符在萬國碼中都有一個唯一的代碼。**